Programação Funcional

Prática a03 - Listas, tuplos, pattern matching e guards

Ficha PL1 — Parte 2 e 3
Cap. 2, pp. 8–19; Cap. 4, pp. 27–37.

Listas

```
[1,2,3]

1 : [2,3]

1 : 2 : 3 : []

[1,2] ++ [3,4]
```

- [ ] → lista vazia
- : → coloca/separa o primeiro elemento
- ++ → concatena duas listas
- Todos os elementos têm de ter o mesmo tipo

head e tail

```
head [1,2,3]
-- 1

tail [1,2,3]
-- [2,3]
```

[1,2,3]

Uma lista não vazia pode ser vista como primeiro elemento + resto da lista.

Tuplos

```
("Ana",20)

fst ("Ana",20)
-- "Ana"

snd ("Ana",20)
-- 20
```

- Uma lista contém vários elementos do mesmo tipo.
- Um tuplo pode juntar elementos de tipos diferentes.

("Ana",20) :: (String, Int)

O que é pattern matching?

Pattern matching permite definir uma função de acordo com a forma dos dados recebidos.

```
primeiroPar :: (a,b) -> a
primeiroPar (x,_) = x
```

- (x,_) corresponde a um par;
- x guarda o primeiro elemento;
- _ significa “este valor não me interessa”.

Pattern matching em listas

```haskell
descricao :: [a] -> String

descricao [] = "Lista vazia"
descricao [_] = "Lista com um elemento"
descricao _ = "Lista com varios elementos"
```

- [ ] → zero elementos
- [_] → exatamente um
- _ → qualquer outro caso

O padrão (x:xs)

Lista: [10,20,30]

x = 10
xs = [20,30]

- (x:xs) corresponde a qualquer lista não vazia.
- x é o primeiro elemento e xs é o resto da lista.

Guards

```haskell
classificaNota :: Int -> String
classificaNota n
    | n < 0     = "Invalida"
    | n < 10    = "Reprovado"
    | n <= 20   = "Aprovado"
    | otherwise = "Invalida"
```

- | → introduz uma guarda
- condição → expressão booleana
- otherwise → caso contrário

Pattern matching ou guards?

Pattern matching: Quando a decisão depende da estrutura dos dados.

```
descricao [] = ...
descricao [_] = ...
```

Guards: Quando a decisão depende de condições sobre valores.

```
classifica n
    | n < 10 = ...
```

Exercícios

Num ficheiro pattern.hs, implementar e testar as seguintes funções:

- ultimoPar — Função que recebe um par e devolve o segundo elemento desse par.
- troca — Função que recebe um par e devolve um novo par com os dois elementos trocados.
- primeiro — Função que recebe uma lista não vazia e devolve o primeiro elemento da lista.
- segundoElemento — Função que recebe uma lista com pelo menos dois elementos e devolve o segundo elemento.
- classificaIdade — Função que recebe uma idade e devolve uma String com a respetiva classificação etária.
- sinal — Função que recebe um número inteiro e devolve 1 se for positivo, 0 se for zero e -1 se for negativo.

```haskell
segundoElemento :: [a] -> a
-- segundoElemento recebe uma lista de elementos de qualquer tipo a
-- e devolve o segundo elemento da lista.

segundoElemento (_:y:_) = y
-- O padrão (_:y:_) exige uma lista com pelo menos dois elementos.
-- O primeiro _ ignora o primeiro elemento.
-- y representa o segundo elemento.
-- O último _ representa o resto da lista, que não é necessário.
-- A função devolve y.
--
-- Exemplo:
-- segundoElemento [10,20,30]
-- y = 20
-- resultado = 20
--
-- Atenção: segundoElemento [] e segundoElemento [x] não estão definidos,
-- porque essas listas não têm um segundo elemento.
```

```haskell
classificaIdade :: Int -> String
-- classificaIdade recebe uma idade representada por um Int
-- e devolve uma String com a classificação dessa idade.

classificaIdade idade
    | idade < 0   = "Idade invalida"
      -- Uma idade negativa é considerada inválida.

    | idade < 13 = "Crianca"
      -- Dos 0 aos 12 anos classificamos como criança.

    | idade < 18 = "Adolescente"
      -- Dos 13 aos 17 anos classificamos como adolescente.

    | idade < 65 = "Adulto"
      -- Dos 18 aos 64 anos classificamos como adulto.

    | otherwise  = "Idoso"
      -- otherwise significa "caso contrário".
      -- Portanto, a partir dos 65 anos classificamos como idoso.
```

Ficha PL1 - Parte 2 e 3

no git https://github.com/v2denny/pf2627

pasta >aulas>a03>FichaPL1.pdf

Desafio

Identificar se uma lista é vazia, unitária ou tem pelo menos dois elementos sem utilizar length.

A função deve receber uma lista e devolver uma String a indicar se a lista é vazia, tem apenas um elemento ou tem pelo menos dois elementos, sem utilizar length.

Exemplos a testar:

```haskell
descricaoLista []
-- "Lista vazia"

descricaoLista [5]
-- "Lista unitaria"

descricaoLista [5,8]
-- "Lista com pelo menos dois elementos"

descricaoLista [1,2,3]
-- "Lista com pelo menos dois elementos"
```